

دانشگاه تهران

پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر

تمرین شماره: ۵
شبکه‌های عصبی و یادگیری عمیق

نام و نام خانوادگی: آرتین توسلی

شماره دانشجویی: ۸۱۰۱۰۲۵۴۳

نیم‌سال دوم
سال تحصیلی ۴۰۴-۴۰۵

فهرست مطالب

۳	Classification Using Tiny Transformer Encoder	۱
۳	تئوری	۱.۱
۳	Self-Attention	۱.۱.۱
۴	Scaled Dot-Product Attention	۲.۱.۱
۵	دلیل تقسیم بر $\sqrt{d_k}$	۳.۱.۱
۷	Multi-Head Attention	۴.۱.۱
۸	Positional Encoding	۵.۱.۱
۸	Encoder, Decoder, Causal Mask	۶.۱.۱
۹	شمارش پارامترهای یک Transformer Encoder Blok	۷.۱.۱
۹	تحلیل Attention در مسئله توییت های بحرانی	۸.۱.۱
۱۰	پیاده سازی	۲.۱
۱۰	پیش پردازش متن و آماده سازی داده	۱.۲.۱
۱۲	Tiny Transformer Encoder از صفر	۲.۲.۱
۱۵	آموزش و ارزیابی	۳.۲.۱
۱۷	بازسازی ساده شده روش مقاله و مقایسه با Tiny Transformer	۴.۲.۱
۱۹	بخش امتیازی	۵.۲.۱

فهرست تصاویر

۱۱	توزیع کلاس‌ها در مجموعه‌های آموزش، اعتبارسنجی و آزمون
۱۶	نمودار تغییرات خطای آموزش و اعتبارسنجی مدل در طول دوره‌ها
۱۶	ماتریس درهم‌ریختگی عملکرد مدل روی داده‌های ارزیابی
۱۸	ماتریس درهم‌ریختگی عملکرد مدل BERT فریز شده روی داده‌های تست
۲۱	نقشه حرارتی توزیع وزن‌های Attention مدل Transformer Tiny

سوال ۱

Classification Using Tiny Transformer Encoder

۱.۱ تئوری

۱.۱.۱ مفهوم کلی Self-Attention

الف) مکانیزم Self-Attention میزان ارتباط و وابستگی هر کلمه از متن را با تمام کلمات دیگر همان متن (فارغ از فاصله آن‌ها در جمله) محاسبه می‌کند. این لایه به جای نگاه کردن به یک پنجره Local، یک دید Global دارد و به مدل اجازه می‌دهد تا با ترکیب اطلاعات کلمات مرتبط، به دید context توکن را نگاه کند.

ب) در مدل‌های RNN (حتی LSTM)، به دلیل پردازش ترتیبی، اطلاعات کلمات ابتدایی در طول مسیر و تا رسیدن به انتهای جملات طولانی کمرنگ می‌شوند و مدل دچار مشکل فراموشی می‌شود. اما در Transformer، فاصله محاسباتی بین دو کلمه در متن وجود ندارد و مدل می‌تواند مستقیماً به هر کلمه دیگری در متن توجه کند. پردازش در RNN‌ها ذاتاً ترتیبی است اما مکانیزم Self-Attention به مدل اجازه می‌دهد تمام کلمات یک جمله را به طور موازی پردازش کند که سرعت آموزش را به شدت افزایش می‌دهد.

ج) در لایه‌های Fully Connected، موقع Inference وزن‌ها ثابت هستند یعنی وزن‌ها وابسته به ورودی تست نیستند. اما در Self-Attention، برای بدست آوردن Attention Score ورودی ضرب در وزن‌هایی می‌شود که خود وابسته به ورودی هستند یعنی میزان توجهی که دو کلمه به هم میکنند بسته به جمله (ورودی) فرق میکند و اینطور نیست بگیم وزن‌های ثابتی هست که در آموزش به آن رسیدیم یعنی α_{ij} تابعی از k_i, v_i, q_i میباشد و خود این‌ها تابعی از ورودی هستند و ما در آموزش فقط W_Q, W_K, W_V را یاد می‌گیریم ولی وزن‌هایی که Attention Score از آن بدست میاد هنوز وابسته به ورودی هست.

$$q_i = x_i W_Q \quad k_i = x_i W_K \quad v_i = x_i W_V$$

(د) چون context مهم است. یک کلمه ممکن است چندین معنی داشته باشد یا صرفاً منظور آن کلمه در آن جمله متفاوت باشد. مثلاً، اگر یکی بنویسد "The movie was a total disaster!"، کلمه "disaster" در متن آمده است اما جمله درباره یک حادثه یا بحران واقعی نیست. Attention به کلمات در این متن با دید context نگاه میکند و میفهمد در این جمله منظور از disaster صرفاً بد بودن فیلم بوده است.

۲.۱.۱ محاسبه Scaled Dot-Product Attention

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$K = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}$$

$$V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 2 & 1 \end{bmatrix}$$

$$d_k = 2$$

(الف)

$$QK^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

(ب)

$$\frac{QK^T}{\sqrt{d_k}} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{bmatrix}$$

(ج) فرمول تابع Softmax برای یک بردار ورودی x (یعنی روی سطر ماتریس ها اعمال میشود چون به هر حال میخوایم برای هر Query کلید هایی که برایش داریم یک توزیع احتمال بشود (چقدر احتمال دارد این کلید برای این Query باشد و مجموع همه این احتمالا برای اون Query یک بشود).

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

حال Softmax را روی ماتریس قسمت ب اعمال کنیم به ازای هر سطرش:

برای سطر اول: $e^{0.707} \approx 2.028$ و $e^0 = 1$. مجموع مخرج می شود $2.028 + 2.028 + 1 = 5.056$. پس وزن های سطر اول تقریباً می شوند: $[0.2, 0.4, 0.4]$ برای سطر دوم: محاسبات مشابه است و وزن ها تقریباً می شوند: $[0.2, 0.4, 0.4]$ بنابراین ماتریس Attention Weights تقریباً برابر است

با:

$$\text{Weights} = \begin{bmatrix} 0.4 & 0.4 & 0.2 \\ 0.2 & 0.4 & 0.4 \end{bmatrix}$$

(د)

$$Z = \begin{bmatrix} 0.4 & 0.4 & 0.2 \\ 0.2 & 0.4 & 0.4 \end{bmatrix} \times \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 0.8 & 1.0 \\ 1.0 & 1.2 \end{bmatrix}$$

جمع وزن دار (که وزنشون را همون وزن attention میگذاریم) از value های کلمات درون جمله.

ه) الان می‌خواهیم نمایش با context از کلمه اول را بسازیم یعنی بینیم Query اول Z اش چقدر میشود. وزن های attention به ما میگوید، مدل چقدر به کلمه اول، دوم و سوم توجه کرده است (مثلاً 40%، 40% و 20%) حال که میدانیم میزان توجه کلمه اول با هر کلمه دیگر این جمله چقدر بوده است، از همین توجه ها میتوان به عنوان وزن در جمع وزن دار گرفت یعنی 40% از ارزش کلمه اول + 40% از ارزش کلمه دوم + 20% از ارزش کلمه سوم یعنی

$$0.4 \times 1 + 0.4 \times 0 + 0.2 \times 2 = 0.8$$

۳.۱.۱ دلیل تقسیم بر $\sqrt{d_k}$

الف) وقتی مقدار d_k بزرگ باشد یعنی تعداد جمع حاصل ضرب هایمان نیز بیشتر هست

$$q \cdot k = \sum_{i=1}^{d_k} q_i k_i$$

حال ممکن است این جمع های بیشتر مقدار کل را نیز بیشتر کند (ممکن هم هست خنثی کند).

ب) چون Softmax نمایی کار میکند، روی اختلاف های کوچک حساس هست و تغییرات زیادی نشان میدهد یعنی اگر score ها خیلی زیاد بششوند، وزن های Attention ای که تولید میکند خیلی شبیه one hot میشود یعنی به یک کلمه خیلی احتمال میدهد (تقریباً 1) و بقیه تقریباً هیچی (0).

ج) الان که Softmax شبیه one hot شده است یعنی هنگام Backpropagation، مشتق از این لایه خیلی به صفر نزدیک میشود.

$$S_i = \frac{e^{x_i}}{\sum_k e^{x_k}}$$

$$\frac{\partial S_i}{\partial x_i} = \frac{e^{x_i} \sum_k e^{x_k} - e^{x_i} e^{x_i}}{(\sum_k e^{x_k})^2}$$

$$\frac{\partial S_i}{\partial x_i} = S_i(1 - S_i)$$

$$\frac{\partial S_i}{\partial x_j} = \frac{0 - e^{x_i} e^{x_j}}{(\sum_k e^{x_k})^2} = - \left(\frac{e^{x_i}}{\sum_k e^{x_k}} \right) \left(\frac{e^{x_j}}{\sum_k e^{x_k}} \right)$$

$$\frac{\partial S_i}{\partial x_j} = -S_i S_j$$

حال فرض کنید $S_i = 1$ میباید و هر j جز i داریم: $S_j = 0$ یعنی همان one hot حال

$$\frac{\partial S_i}{\partial x_j} = 0$$

$$\frac{\partial S_i}{\partial x_i} = 0$$

و یعنی به طور شدید دچار vanishing gradient میشود و وزن های شبکه آپدیت نمیشود و یادگیری سخت میشود.

د) با این scaling، در اصل ما تونستیم بعد مدل d_k را از واریانس حذف کنیم. واریانس تقسیم بر $\sqrt{d_k}$ باعث پایدار شدن فرآیند می شود به این خاطر که توزیع خروجی Attention را از سایز و ابعاد مدل مستقل می کند. فرقی نمی کند d_k برابر 10 باشد یا 1000؛ اعدادی که وارد Softmax می شوند همیشه پراکندگی معقول و ثابتی دارند. Softmax در ناحیه ی نرم خودش کار می کند و از vanishing gradient جلوگیری میشود.

فرض کنید درایه های بردارهای q و k متغیرهای تصادفی مستقلی با میانگین 0 و واریانس 1 باشند:

$$E[q_i] = 0, \quad Var(q_i) = 1$$

$$E[k_i] = 0, \quad Var(k_i) = 1$$

$$S = q \cdot k = \sum_{i=1}^{d_k} q_i k_i$$

$$E[q_i k_i] = E[q_i] E[k_i] = 0 \times 0 = 0$$

$$Var(q_i k_i) = E[(q_i k_i)^2] - (E[q_i k_i])^2$$

$$Var(q_i k_i) = E[q_i^2] E[k_i^2] - 0$$

از آنجا که واریانس q_i و k_i برابر 1 و میانگین آنها 0 است، پس $E[q_i^2]$ و $E[k_i^2]$ نیز همان 1 هستند:

$$Var(q_i k_i) = 1 \times 1 = 1$$

$$Var(S) = Var\left(\sum_{i=1}^{d_k} q_i k_i\right) = \sum_{i=1}^{d_k} Var(q_i k_i) = \sum_{i=1}^{d_k} 1 = d_k$$

$$Var\left(\frac{S}{\sqrt{d_k}}\right) = \frac{1}{(\sqrt{d_k})^2} Var(S) = \frac{1}{d_k} \times d_k = 1$$

۴.۱.۱ شکل تنسورها در Multi-Head Attention

$$batch_size = 32$$

$$sequence_length = 40$$

$$d_model = 64$$

$$num_heads = 4$$

(الف)

$$d_{head} = \frac{d_{model}}{num_heads} = \frac{64}{4} = 16$$

(ب) در ابتدا، ورودی لایه Attention (با ابعاد $32 \times 40 \times 64$) در ماتریس‌های وزن W_V و W_K ، W_Q ضرب می‌شود و این ماتریس‌ها ابعادشون $d_{model} \times d_{model}$ (یعنی 64×64) است پس شکل تنسورهای Q ، K و V برابر $32 \times 40 \times 64$ میشود.

(ج) برای اینکه محاسبه Attention در تمام Headها به صورت موازی انجام شود، بعد آخر (d_{model}) را به دو بعد مجزای d_{head} و num_heads می‌شکنیم (عملیات Reshape). سپس معمولاً جای بُعدهای دوم و سوم را عوض می‌کنیم تا بعد Headها دقیقاً در کنار Batch قرار بگیرد. بنابراین شکل ماتریس‌ها:

$$(batch_size, num_heads, sequence_length, d_{head}) = (32, 4, 40, 16)$$

(د) وقتی $Q \times K^T$ برای یک جمله مشخص در بیج و یک Head مشخص، ابعاد Q برابر $(40, 16)$ و ابعاد K^T برابر $(16, 40)$ است. حاصل ضرب $(40, 40)$ حال برای همه جملات در بیج و هد ها، ابعاد قبل و بعد Softmax:

$$Shape = (batch_size, num_heads, sequence_length, sequence_length) = (32, 4, 40, 40)$$

(ه) حال وقتی این وزن‌های Attention را در V ضرب می‌کنیم ابعادش همیشه $(32, 4, 40, 16)$ دارد (دوباره مثل قبل اول فرض کنید برای یک جمله در بیج و هد مشخص داریم حل می‌کنیم و یعنی $40, 40$ ضرب در $40, 64$ که میشه $40, 64$ و بعد تعمیم میدیم برای همه جملات در بیج و همه هد ها). برای ترکیب آن‌ها، ابتدا ابعاد را جابه‌جا می‌کنیم (Transpose به حالت $(32, 40, 4, 16)$ و سپس در بُعد آخر آن‌ها را به هم Concatenate می‌کنیم.

$$Shape = (batch_size, sequence_length, num_heads \times d_{head}) = (32, 40, 64)$$

این تنسور وقتی بعداً در ماتریس خروجی ضرب میشه باز ابعادش همین می‌موند.

(و) نمی‌توانیم بردار 64 بعدی را به قسمت‌های کاملاً مساوی تقسیم کنیم و در مرحله Reshape کردن تنسورها با خطای عدم تطابق ابعاد مواجه می‌شویم.

۵.۱.۱ توضیح Positional Encoding

الف) چون اصولاً در این روش ما صرفاً به خود کلمات اهمیت می‌دهیم و شبیه یک مجموعه از کلمات دیده می‌شود و نه ترتیب آنها یعنی attention دو کلمه به هم اگر جایشون را در جمله عوض کنیم باز در فرمول QK^T همان مقدار قبلی را می‌دهد (صرفاً جاش تو ماتریس تغییر می‌کند).

ب) هر چند معنی کلمه خیلی مهم هست، مکان آن نیز مهم هست و ما از PE استفاده می‌کنیم که این اطلاعات مکانی را به خود بردار کلمه اضافه کنیم و اینطور هم موقعیت مطلق (کجای جمله هست) و هم موقعیت نسبی (فاصله اش با دیگر کلمات) را می‌تواند بفهمد و قدرت مدل را بیشتر می‌کند و دقیق ترش می‌کند.

ج) pos نشون می‌دهد در کجای جمله قرار داریم (برای کلمه اول 0 هست و برای کلمه دوم 1 و ...). i فرکانس موج مثلثی را تعیین می‌کند (برای بعد های اول که i کم هست، فرکانس زیاد هست و مقادیر به سرعت تغییر می‌کند و برای بعد های انتهایی این تغییرات کند می‌شود). d_{model} هم که طول بردار ویژگی مدل هست که باعث می‌شود توزیع فرکانس‌ها به درستی در تمام طول بردار پخش شود.

د) پارامتر اضافی برای یادگیری ندارد و همینطور در سینوسی ما برای هر طولی از جمله می‌توانیم embedding مکانی اش را بدست بیاریم مثلاً در داده آموزش فقط جملات به طول 16 دیده شده ولی در تست جمله ای به طول 64 بیاد ولی برای مدلای یادگیری چون چنین طولی را ندیده است، صرفاً مثلاً 16 تا پارامتر به ما داده است که چطور 16 تا کلمه اول را مکانش را embed کنیم ولی برای این مثال محدودیتش معلوم هست.

هـ) ساختار جمله مثلاً نهاد و فاعل و مفعول و ... را از دست می‌دهد و مفهوم جمله هم یعنی می‌تواند خیلی تغییر کند برای مثال Artin hit Borna و Borna hit Artin فقط ترتیب کلمات عوض شده است ولی معنی جمله کامل تغییر کرده است.

۶.۱.۱ Encoder, Decoder, Causal Mask

الف) انکودر وظیفه اش این هست که یک نمایش برداری غنی و با توجه به context برای تک تک کلمات تولید می‌کند و برای اینکار منطقاً باید متن را درک کند و ویژگی های مهم آن را استخراج بکند. حال دیکودر برای generation استفاده می‌شود یعنی به صورت Autoregressive (فقط با دید به کلمات عقب تر از کلمه فعلی)، کلمات را یکی یکی تولید می‌کند (یک Cross-Attention هم دارد تا به خروجی های انکودر نگاه کند و بفهمد چه چیزی باید تولید کند).

ب) همانطور که در بالاتر گفتیم Autoregressive هست و چون نمی‌خواهیم آموزش ترتیبی و کند باشد و می‌خواهیم یک دفعه کل جمله را بدهیم، برای اینکه مدل از کلمات جلوتر استفاده نکند برای تولید کلمه ای، ما به سادگی از ایده ماسک استفاده می‌کنیم. این ماسک، Attention هر کلمه را با کلمات آینده قطع می‌کند (با قرار دادن مقدار منفی بی نهایت قبل از سافت مکس). در نتیجه، مدل مجبور می‌شود کلمه بعدی را فقط و فقط بر اساس کلمات گذشته و حال پیش بینی کند (دقیقاً مشابه شرایط زمان Inference)

ج) خیر چون generation نیست مسئله و یک مسئله classification هست و با کمک انکودر می‌توانیم ما متن را درک کنیم و صرفاً با یک classifier head بعد از اینکه متن را درک کردیم نظر هم می‌توانیم دربارش بدیم (در انکودر مدل را کور نمی‌کنیم چون تاثیرات برعکس ممکن است بزند و معنی جمله را اشتباه بفهمد مثلاً Fire, this movie was great اینجا باید جلوترش را ببیند تا بفهمد به منظور خوب بودن فیلم استفاده شده است و آتشی در واقعیت وجود ندارد).

د) همانطور که بالاتر توضیح دادم، انکودر ویژگی های مهم متن را فهمیده است و مفهوم را گرفته است و ما به دنبال تولید متنی نیستیم بلکه صرفاً می‌خواهیم روی همین ویژگی ها تصمیم بگیریم آیا برچسپ 0 بدیم یا 1 که با یک classifier head ساده بر انکودر می‌توان اینکار را بکنیم (با کمک یک توکن [CLS] که

نماینده کل جمله است و به این head داده میشود.)

۷.۱.۱ شمارش پارامترهای یک Transformer Encoder Blok

الف) ۴ ماتریس Projection داریم: W_Q, W_K, W_V, W_O ابعاد هر کدام از این ماتریس ها 64×64 است.

تعداد پارامترهای هر ماتریس: $64 \times 64 = 4096$ تعداد کل پارامترهای این بخش: $4 \times 4096 = 16384$

(ب)

پارامترهای $W_1: 64 \times 128 = 8192$ پارامترهای $W_2: 128 \times 64 = 8192$ تعداد کل پارامترهای Feed Forward Network برابر است با:
 $8192 + 8192 = 16384$

(ج)

بخش MHA: چهار ماتریس داریم که بُعد خروجی هر چهار تای آن ها ۶۴ است. بایاس های این بخش: $4 \times 64 = 256$ بخش FFN: بُعد خروجی ماتریس W_1 برابر با ۱۲۸ است. بُعد خروجی ماتریس W_2 برابر با ۶۴ است. مجموع بایاس های این بخش: $128 + 64 = 192$ تعداد کل بایاس ها: $256 + 192 = 448$ (ب) به ماتریس های Projection مثلا نگاه کنید. ابعاد این ماتریس ها $d_{model} \times d_{model}$ هستند، این یعنی تعداد پارامتر ها با $O(d_{model}^2)$ رشد میکند (در قسمت ب و ج هم باز افزایش پارامتر خواهیم داشت ولی در کل قسمت الف خیلی ساده نشان میدهد که ابعاد مدل را زیاد کنیم به صورت توان دو پارامتر ها زیاد میشود.)

ه) تعداد پارامترهای بخش Encoder دو برابر می شود و یعنی زمان اجرای یک Forward Pass و Backward Pass تقریباً دو برابر می شود. در نتیجه، سرعت پردازش هر اپیک کمتر میشود و احتمال Overfitting هم بیشتر میشود (مانند هر مدلی که تا الان دیدیم، اگر ظرفیت مدل را بیشتر کنیم بدون اینکه تعداد داده رو زیاد کنیم، باعث میشود مدل بیشتر به سمت حفظ کردن الگو ها برود تا یادگیری الگوهای پیچیده بخاطر همین overfit میشود.)

۸.۱.۱ تحلیل Attention در مسئله توییت های بحرانی

الف) کلماتی که بیشتر بحران وضعیت را دارند نشان میدهند یعنی Emergency, Fire و در یک لول کمتر building, responding, و در یک لول کمتر crews, downtown و در یک لول کمتر توجه مثلا a, to, are (اگر مدل نگه داشته بود). البته اینا حدس های بنده است و مدل ممکن است کلمه ای را مهمتر دونسته باشد.

ب) خیر چون از این کلمه در موقعیت های غیر بحرانی هم میشود استفاده کرد مثلا My outfit is fire و حتی اگر در همون معنا بود ممکن بود مثلا not on fire باشد که یعنی آتشی رخ نداده است ولی اگر فقط فقط به fire توجه کنیم، میگیریم بحرانی رخ داده است.

ج) مدل مانند اول مثل ما کلمات را جدا میگیرد و بعد که باهوش تر میشود میتواند به ترکیب کلمات توجه کند که بعضا حتی ممکن است معنی آنها تغییر کند مثلا شاید اول فکر کند Emergency crews مهمتر از crews هست و بعدا که باهوش تر میشود (در لایه های جلوترش)، میفهمد Emergency crews یعنی نیرو های امدادی که موقعیت های بحرانی خیلی مهم هست. Building fire هم یک خطر را نشان میدهد و responding هم که واکنش میشود و با کنار گذاشتن اینا در آخرین لایه هایش میفهمد که نیرو های امدادی دارند واکنشی میدهند نسبت به یک خطری و بعد هم این را به عنوان بحران طبقه بندی میکند.

(د) قسمت ب را ببینید.

۲.۱ پیاده‌سازی

۱.۲.۱ پیش‌پردازش متن و آماده‌سازی داده

برای پاک‌سازی متن، تابعی به نام `clean_tweet_text` پیاده‌سازی شد که دقیقاً از روند پیشنهادی مقاله مرجع پیروی می‌کند. عملیات زیر به ترتیب روی متن هر توییت اعمال کردم:

- کوچک کردن حروف (Case Normalization): تمامی حروف به حالت کوچک تبدیل شدند تا از ایجاد توکن‌های تکراری برای کلمات مشابه جلوگیری شود.
- حذف مقادیر نامرتبط: ایمیل‌ها، آدرس‌های اینترنتی (URLs) و تگ‌های HTML از متن حذف شدند.
- حذف ایموجی‌ها: با استفاده از کتابخانه `emoji`، تمامی شکلک‌ها از متن پاک‌سازی شدند.
- بسط مخفف‌ها (Abbreviations): یک دیکشنری جامع از مخفف‌های رایج در زبان انگلیسی و شبکه‌های اجتماعی تهیه شد تا کلماتی مانند "omg" یا "dnt" به معادل‌های کامل خود بسط داده شوند.
- حذف کاراکترهای ویژه: کاراکترهای غیرحرفی و علائم نگارشی تکراری حذف شدند تا تنها کلمات معنادار باقی بمانند.
- حذف کلمات توقف (Stopwords): با استفاده از کتابخانه `nlk`، کلمات توقف انگلیسی که ارزش معنایی پایینی در تشخیص بحران دارند، از متن حذف گردیدند.

برای اطمینان از تکرارپذیری نتایج در مراحل بعدی، یک تابع `set_seed` پیاده‌سازی شد تا تمامی مقادیر تصادفی در `torch`، `numpy` و هسته پایتون روی عدد ثابت 42 تنظیم شوند.

پیش از اعمال پاک‌سازی روی کل مجموعه داده، وضعیت مقادیر از دست رفته در ستون‌های مختلف بررسی شد. نتایج خروجی به شرح زیر است:

نام ستون	تعداد مقادیر از دست رفته	درصد از کل داده‌ها
location	3418	30.06%
id	0	0.00%
keyword	0	0.00%
text	0	0.00%
target	0	0.00%

حذف ستون `location`: با توجه به جدول بالا، بیش از 30 درصد از داده‌های مربوط به موقعیت مکانی ناموجود هستند. پرکردن این حجم از داده‌های از دست رفته با مقادیر ساختگی می‌تواند به مدل بایاس وارد کند، تصمیم گرفته شد که این ستون به طور کامل از فرآیند آموزش حذف شود.

پاک‌سازی تهاجمی متن: با وجود اینکه حذف کلمات توقف و ایموجی‌ها ممکن است بخش کوچکی از لحن کاربر را از بین ببرد، اما بجاش توجه مدل متمرکز تر میشود و اندازه Vocabulary هم کمتر میکند و به مدل کمک می‌کند تا با توجه به محدودیت داده‌ها، پارامترهای خود را روی کلمات کلیدی و مهم‌تر (که نشان‌دهنده بحران واقعی هستند) متمرکز کند.

پیش از تقسیم داده‌ها، ساختار ورودی به گونه‌ای تغییر یافت که کلمه کلیدی (`keyword`) نیز در کنار متن اصلی توییت قرارگیرد. فرمت نهایی هر متن به

شکل زیر در آمد: [CLS] keyword [SEP] clean_text

سپس کل مجموعه داده با نسبت‌های 80% برای آموزش (Train)، 10% برای اعتبارسنجی (Validation) و 10% برای آزمون (Test) تقسیم شد. برای جلوگیری از بهم خوردن توزیع کلاس‌ها در مجموعه‌های کوچکتر، از پارامتر stratify بر اساس برچسب هدف استفاده گردید.

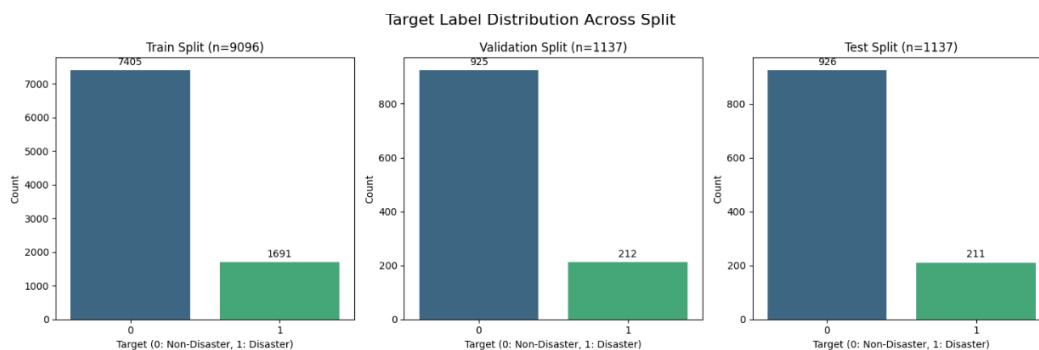
برای تبدیل کلمات به شناسه‌های عددی (Token IDs)، کلاس Vocabulary پیاده‌سازی شد.

تنها با استفاده از داده‌های بخش آموزش ساخته شد تا data leakage رخ ندهد. علاوه بر توکن‌های [CLS] و [SEP]، دو توکن ضروری <PAD> (با شناسه 0) برای هم‌طول کردن توالی‌ها و <UNK> (با شناسه 1) برای مدیریت کلمات خارج از دیکشنری که ممکن است در داده‌های اعتبارسنجی یا آزمون دیده شوند، به دیکشنری اضافه شدند.

از آنجایی که تنسورهای با ابعاد ثابت در هر Batch می‌خواهیم، طول تمامی توالی‌ها با استفاده از تابع pad_X روی عدد ثابت SEQ_MAX_LEN = 50 تنظیم شد.

- توالی‌های بلندتر از 50 توکن، بریده (Truncate) شدند.
- توالی‌های کوتاه‌تر از 50 توکن، با استفاده از شناسه توکن <PAD> پر شدند.

توزیع کلاس‌ها در سه مجموعه آموزش، اعتبارسنجی و آزمون بررسی شد. نتایج:



شکل ۱.۱: توزیع کلاس‌ها در مجموعه‌های آموزش، اعتبارسنجی و آزمون

مجموعه داده	توزیع کلاس ۰ (Non-Disaster)	توزیع کلاس ۱ (Disaster)	تعداد کل
Train	81.41%	18.59%	9096
Validation	81.35%	18.65%	1137
Test	81.44%	18.56%	1137

همان‌طور که از نتایج پیداست، دیتاست دارای عدم تقارن (Class Imbalance) قابل توجهی است؛ به طوری که بیش از 80 درصد داده‌ها متعلق به کلاس 0 (توییت‌های غیربحرانی) و کمتر از 20 درصد متعلق به کلاس 1 (بحران واقعی) هستند.

استفاده از روش Stratified Sampling در زمان تقسیم داده‌ها تضمین کرده است که این عدم تقارن در هر سه مجموعه آموزش، اعتبارسنجی و آزمون حفظ شود. اگر تقسیم‌بندی به صورت کاملاً تصادفی انجام می‌شد، ممکن بود مجموعه اعتبارسنجی یا آزمون دارای توزیع متفاوتی باشند که این امر ارزیابی نهایی مدل را نامعتبر می‌ساخت.

۲.۲.۱ پیاده‌سازی Tiny Transformer Encoder از صفر

در ابتدا باید اعداد صحیح (شناسه توکن‌ها) را به بردارهایی با ابعاد d_{model} تبدیل کنیم.

از کلاس nn.Embedding در پای‌تورچ استفاده شد. خروجی این لایه در $\sqrt{d_{model}}$ ضرب می‌شود. این کار باعث می‌شود تا واریانس مقادیر Embedding افزایش یابد و وقتی با Positional Encoding ترکیب می‌شود، تأثیر وزن‌های معنایی نسبت به وزن‌های موقعیتی کم‌رنگ نشود.

$$Input = \text{Embedding Token} \times \sqrt{d_{model}} + \text{Encoding Positional}$$

ابعاد لایه Embedding:

- ورودی: یک تانسور دو بعدی با ابعاد ($batch_size = 32, sequence_length = 50$)
- خروجی: یک تانسور سه بعدی با ابعاد ($batch_size = 32, sequence_length = 50, embedding_dim = 64$)

یعنی در یک جمله مشخص، یک توکن مشخص‌اش را به توسط یک بردار 64 بعدی نمایش می‌دهیم.

برخلاف شبکه‌های بازگشتی (RNN) که ذاتاً ترتیب کلمات را درک می‌کنند، مکانیزم Self-Attention در ترنسفورمرها به صورت موازی عمل کرده و هیچ درکی از موقعیت کلمات نسبت به یکدیگر ندارد. برای حل این مشکل، لایه PositionalEncoding پیاده‌سازی شد. در این پیاده‌سازی، از توابع سینوسی و کسینوسی با فرکانس‌های متفاوت برای تولید بردارهای موقعیت استفاده گردید.

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

برای جلوگیری از آموزش این لایه در فرآیند Backpropagation، بردارهای تولید شده با استفاده از متد register_buffer به عنوان یک استیت ثابت در ماژول ثبت شدند. خروجی این لایه (یک تانسور 3 بعدی) دقیقاً ابعادی برابر با خروجی لایه Embedding دارد و در متد forward، این دو مقدار به صورت عنصر به عنصر با یکدیگر جمع می‌شوند.

در نهایت در ModelInput ورودی انکودر را درست می‌کنیم (از token embedding و بردار positional encoding را عبور می‌دهیم و بعدش با positional encoding جمع می‌کنیم و برای اینکه overfit هم نشود از یک لایه Dropout استفاده کردیم).

در نهایت خروجی این را (با ابعاد [64, 50, 32])، وارد لایه Multi-Head Self-Attention می‌کنیم. مکانیزم Self-Attention که میزان ارتباط هر کلمه با سایر کلمات موجود در جمله را محاسبه می‌کند را در این مرحله پیاده‌سازی می‌کنیم. در این بخش، کلاس ScaledDotProductAttention پیاده‌سازی شد که عملیات زیر را به ترتیب روی ورودی‌های Q (Query)، K (Key) و V (Value) انجام می‌دهد:

- محاسبه امتیازات خام (Unscaled Scores): از طریق ضرب ماتریسی Query در Key (ترنسپوزاش) به دست می‌آید.

- مقیاس‌بندی (Scaling): برای جلوگیری از رشد بی‌رویه مقادیر و ورود تابع Softmax به نواحی با گرادیان صفر، امتیازات خام بر $\sqrt{d_k}$ تقسیم می‌شوند.
- اعمال Mask: برای جلوگیری از توجه مدل به توکن‌های <PAD> (که صرفاً برای هم‌طول‌سازی اضافه شده‌اند و ارزش معنایی ندارند)، درایه‌های مربوط به این توکن‌ها در ماتریس امتیازات با عدد بسیار کوچکی ($-1e9$) جایگزین می‌شوند تا پس از اعمال Softmax، وزن آن‌ها صفر شود.
- تولید وزن‌های توجه و خروجی نهایی: با اعمال تابع Softmax روی امتیازات مقیاس‌شده، وزن‌های Attention تولید شده و در نهایت در ماتریس Value ضرب می‌شوند.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

حال می‌خواهیم از چندین Head برای Attention استفاده کنیم یعنی چندین Head مستقل و موازی بتواند به صورت هم‌زمان به جنبه‌های مختلف معنایی و نحوی کلمات توجه کند چون خیلی مهم هست موازی باشند و نه ترتیبی، از for استفاده نشده است و بجاش با کمک تغییر شکل تنسور ها توانستیم موازی سازی کنیم:

- B: اندازه دسته یا Batch Size (32)
- S: طول توالی یا Sequence Length (50)
- D: بعد Embedding یا d_{model} (64)
- H: تعداد Head ها (4)
- d_k : بعد هر Head که برابر با $D/H = 16$ است.

ابتدا ورودی X در ماتریس‌های وزن قابل یادگیری ضرب می‌شود تا Q، K و V تولید شوند:

$$Q = XW^Q$$

$$K = XW^K$$

$$V = XW^V$$

ابعاد: $[B, S, D] \rightarrow [32, 50, 64]$

به جای برش ماتریس، بعد D به دو بعد مجزای H و d_k تغییر شکل میدیم:

ابعاد: $[B, S, H, d_k] \rightarrow [32, 50, 4, 16]$

ابعاد Sequence (S) و Head (H) جابجا (Transpose) می‌شوند. با آوردن بعد Head به اندیس 1، تابع matmul در PyTorch ابعاد B و H را به عنوان ابعاد مستقل دسته (Batch) در نظر می‌گیرد و ضرب ماتریسی را صرفاً روی دو بعد آخر (S و d_k) برای تمامی $32 \times 4 = 128$ ماتریس مجزا به صورت هم‌زمان انجام می‌دهد.

ابعاد: $[B, H, S, d_k] \rightarrow [32, 4, 50, 16]$

محاسبات مربوط به ScaledDotProductAttention انجام می‌شود:

• ابعاد Q: $[32, 4, 50, 16]$

• ابعاد

K^T : $[32, 4, 16, 50]$

- Scores : $[32, 4, 50, 50]$
- $\text{softmax}(\text{Scores}) \times V$: $[32, 4, 50, 16]$

برای کنار هم قرار دادن مجدد Headها، عمل Transpose معکوس می‌شود تا Sequence به اندیس 1 بازگردد:

ابعاد: $[B, S, H, d_k] \rightarrow [32, 50, 4, 16]$

سپس با استفاده از تابع $\text{view}()$ ، ابعاد H و d_k مجدداً در هم ترکیب شده و به بعد D تبدیل می‌شوند:

$\text{Concat}(\text{head}_1, \dots, \text{head}_H)$

ابعاد: $[B, S, D] \rightarrow [32, 50, 64]$

در نهایت، خروجی در ماتریس وزن نهایی ضرب می‌شود:

$Z = \text{Concat}(\dots)W^z$

ابعاد خروجی نهایی مدل (Z): $[32, 50, 64]$

پس از استخراج ویژگی‌های مبتنی بر توجه در لایه Multi-Head Attention، خروجی به یک شبکه Feed Forward ارسال می‌شود. این شبکه به صورت مستقل روی هر موقعیت از توالی اعمال می‌شود و هدف اصلی آن، افزایش ظرفیت غیرخطی مدل است. در این پیاده‌سازی، این لایه را با یک MLP ساختم که ابعاد تنسور ورودی از d_{model} (64) به d_{hidden} (128) بسط داده شده، از تابع فعال‌ساز ReLU و یک لایه Dropout عبور کرده و مجدداً به بعد اصلی d_{model} تصویر می‌شود تا با ابعاد سایر لایه‌ها سازگار بماند.

در این بخش، لایه‌های MultiHeadAttention و FeedForward در قالب یک بلوک کامل انکودر (Encoder Block) با یکدیگر ترکیب شدند. برای جریان بهتر گرادینت‌ها و جلوگیری از Vanishing Gradients، از اتصالات میان‌بر استفاده شده است.

در معماری ترنسفورمر، دو رویکرد اصلی برای جایگذاری لایه LayerNorm وجود دارد: Pre-LN و Post-LN. در این پیاده‌سازی از ساختار Post-LN

استفاده شده است. در این فرم، خروجی هر زیرلایه ابتدا با ورودی همان زیرلایه جمع شده و سپس نرمال‌سازی روی حاصل جمع اعمال می‌شود:

$$\text{Output} = \text{LayerNorm}(x + F(x))$$

در مقابلش Pre-LN وجود دارد که به فرم:

$$\text{Output} = x + \text{LayerNorm}(F(x))$$

این رویکرد نسبت به Pre-LN در برخی سناریوها به پایداری بهتری در انتهای فرآیند آموزش منجر می‌شود (هرچند پایداری آموزش Pre-LN معمولاً بهتر است چون در ابتدای آموزش گرادیان هایش رفتار بهتری دارند ولی ما طبق همان ترجیح صورت پروژه جلو رفتیم).

تمامی اجزای پیاده‌سازی شده در مراحل قبل، در قالب کلاس نهایی TinyTransformerClassifier سرهم‌بندی شدند. در این معماری از $N = 2$ بلوک انکودر به صورت متوالی استفاده شده است.

حال باید ببینیم خروجی انکودر که برای یک جمله 50 بردار 64 بعدی می‌باشد چطور به یک بردار واحد برای طبقه بندی تبدیل کنیم. من در این پیاده سازی از استراتژی CLS Token Pooling استفاده کردم. از آنجا که در مرحله پیش‌پردازش، توکن ویژه [CLS] در ابتدای تمامی جملات (اندیس 0) قرار داده شده بود، مدل یاد می‌گیرد که اطلاعات کلیدی کل متن را در بردار خروجی مربوط به همین توکن جمع کند. بنابراین، تنها تانسور مربوط به $x[:, 0, :]$ استخراج شده و به یک Classifier (شامل لایه‌های Linear، ReLU و Dropout) داده می‌شود تا لاجیت‌های مربوط به دو کلاس خروجی را تولید کند.

با توجه به بررسی‌های اولیه روی دیتاست که نشان‌دهنده توزیع نامتقارن برجسته‌ها (حدود 81% کلاس 0 و 19% کلاس 1) بود، آموزش مستقیم مدل می‌توانست به سوگیری شدید به سمت کلاس اکثریت منجر شود. برای حل این مشکل، اقدامات زیر در مرحله تعریف تابع هزینه و بهینه‌ساز انجام شد:

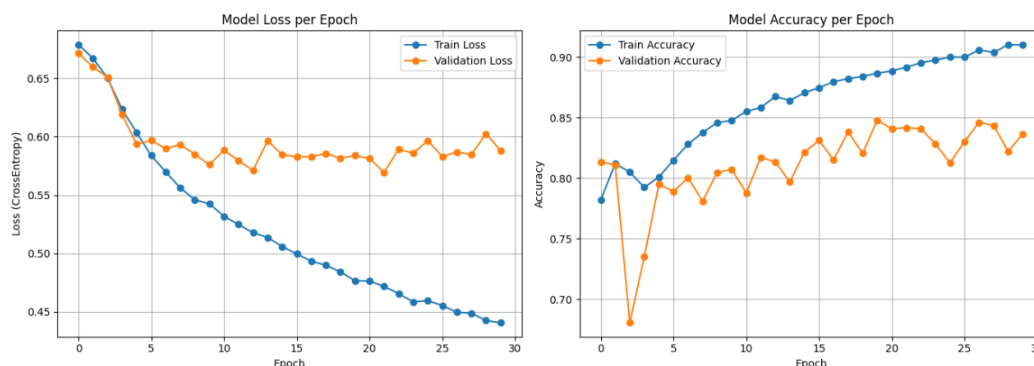
- وزن‌دهی به کلاس‌ها: با استفاده از معکوس جذر تعداد نمونه‌های هر کلاس، وزن‌های متناسبی محاسبه شد. بر این اساس، کلاس 0 (غیربحرانی) وزن 0.3234 و کلاس 1 (بحرانی) وزن 0.6766 را دریافت کرد. این مقادیر به عنوان پارامتر weight به CrossEntropyLoss پاس داده شدند تا خطای تشخیص اشتباه در کلاس اقلیت، جریمه سنگین‌تری داشته باشد.
- برای جلوگیری از اعتماد بیش از حد مدل (Overconfidence) به پیش‌بینی‌های خود، پارامتر $\text{label_smoothing} = 0.1$ اعمال گردید.
- از الگوریتم Adam با نرخ یادگیری 10^{-4} و همچنین پارامتر weight_decay برای اعمال Regularization (L2) استفاده شد.

۳.۲.۱ آموزش و ارزیابی

مدل برای 30 دوره (Epoch) آموزش دید. با بررسی نمودارهای دقت و هزینه می‌فهمیم که:

خطای آموزش به صورت پیوسته و یکنواخت تا انتهای فرآیند کاهش یافته و دقت آموزش به عدد بالای 91% رسیده است. این موضوع نشان می‌دهد که ظرفیت مدل برای یادگیری الگوهای موجود در داده‌های آموزشی کاملاً کافی بوده است. خطای اعتبارسنجی در دوره‌های ابتدایی همگام با خطای آموزش کاهش می‌یابد، اما از حوالی دوره 20 به بعد، روند کاهشی آن متوقف شده و نوسانات افزایشی از خود نشان می‌دهد. این نقطه، آغاز پدیده Overfitting را نشان می‌دهد؛ جایی که مدل به جای یادگیری مفاهیم کلی، شروع به حفظ کردن نویزهای مجموعه آموزش می‌کند. با توجه به ذخیره‌سازی مدل بر اساس کمترین خطای اعتبارسنجی (در کد)، بهترین وزن‌های مدل (مربوط به دوره 22) بارگذاری شد تا از تبعات Overfitting در ارزیابی نهایی جلوگیری شود.

پس از بارگذاری بهترین مدل، عملکرد آن روی دو مجموعه Validation و Test بر اساس معیارهای استاندارد دسته‌بندی محاسبه شد. نتایج در جدول

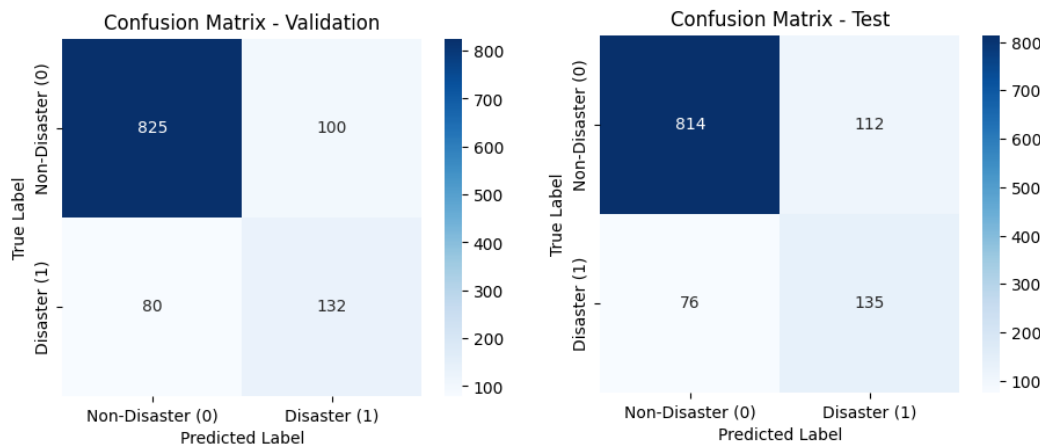


شکل ۲.۱: نمودار تغییرات خطای آموزش و اعتبارسنجی مدل در طول دوره‌ها

زیر خلاصه شده‌اند (کلاس 1 یعنی disaster مثبت گرفته شده است):

مجموعه تست	مجموعه اعتبارسنجی	معیار ارزیابی (Metric)
0.8347	0.8417	Accuracy
0.5466	0.5690	Precision
0.6398	0.6226	Recall
0.5895	0.5946	F_1-score

بررسی ماتریس‌های درهم‌ریختگی :



شکل ۳.۱: ماتریس درهم‌ریختگی عملکرد مدل روی داده‌های ارزیابی

عملکرد در مجموعه تست: مدل توانسته 814 نمونه غیربحرانی (TN) و 135 نمونه بحرانی واقعی (TP) را به درستی تشخیص دهد. مدل 112 نمونه عادی را به اشتباه بحران تشخیص داده است (هشدار کاذب). مدل 76 بحران واقعی را نادیده گرفته و عادی دسته‌بندی کرده است. در بخش قبل، برای مقابله با توزیع نامتقارن داده‌ها (81% کلاس 0 در برابر 19% کلاس 1)، وزن کلاس بحران در تابع هزینه بیشتر در نظر گرفته شد ($weight_1 = 0.6766$). معیار Recall (حدود 0.64) به شکل معناداری از معیار Precision (حدود 0.55) بالاتر است. در مسائل تشخیص بحران، این یک رفتار مطلوب است؛ زیرا از دست دادن یک گزارش بحران واقعی هزینه و خطرات جانی بسیار بیشتری نسبت به بررسی یک هشدار کاذب دارد.

۴.۲.۱ بازسازی ساده‌شده روش مقاله و مقایسه با Tiny Transformer

برخلاف مدل Tiny Transformer که در آن یک لغت‌نامه ساده از صفر ساخته شد، مدل‌های پیش‌آموزش‌دیده نیازمند توکنایزر اختصاصی خود هستند تا متون را دقیقاً با همان Vocabulary و ساختاری که در زمان Pre-training دیده‌اند، پردازش کنند. برای این کار از کلاس BertTokenizer استفاده شد.

در بخش قبل، کلمات کلیدی (keyword) به صورت دستی با استفاده از تگ‌های [CLS] و [SEP] به متن چسبانده شده بودند. اما ارسال این متن دستکاری‌شده به توکنایزر BERT باعث ایجاد رفتار غیرعادی و دوگانگی در توکن‌های ویژه می‌شود (برای مثال تولید توکن‌های تکراری مانند [CLS] [CLS]). برای رفع این مشکل، از قابلیت ذاتی توکنایزر BERT در پردازش جملات جفتی استفاده شد. کلمه کلیدی به عنوان Text A و متن پاک‌سازی‌شده به عنوان Text B به صورت همزمان به توکنایزر پاس داده شدند. با این کار، توکنایزر به صورت خودکار و کاملاً استاندارد، توالی را به فرمت زیر قالب‌بندی کرد: [CLS] keyword [SEP] clean_text [SEP]

علاوه بر شناسه‌های عددی (input_ids)، توکنایزر یک ماسک توجه (attention_mask) نیز تولید کرد تا به مدل نشان دهد کدام بخش‌ها توکن واقعی و کدام بخش‌ها پدینگ (صفر) هستند.

برای ساخت مدل طبقه‌بند، از کلاس BertForSequenceClassification استفاده شد. این مدل به گونه‌ای طراحی شده است که نمایش نهایی توکن [CLS] را از لایه‌های عمیق ترنسفورمر دریافت کرده و آن را از یک لایه خطی (Classifier Head) عبور می‌دهد تا لاجیت‌های مربوط به دو کلاس (بحران و غیربحران) تولید شوند.

به منظور کاهش چشمگیر زمان اجرا و جلوگیری از تخریب وزن‌های از پیش‌آموزش‌دیده در اثر آموزش روی یک دیتاست کوچک، وزن‌ها را freeze می‌کنیم. تمامی لایه‌های پایه‌ای و استخراج‌کننده ویژگی مدل BERT ثابت نگه داشته شدند (بدون محاسبه گرادیان) و صرفاً لایه نهایی طبقه‌بندی آموزش داده شد.

- تعداد کل پارامترهای مدل: 109483778

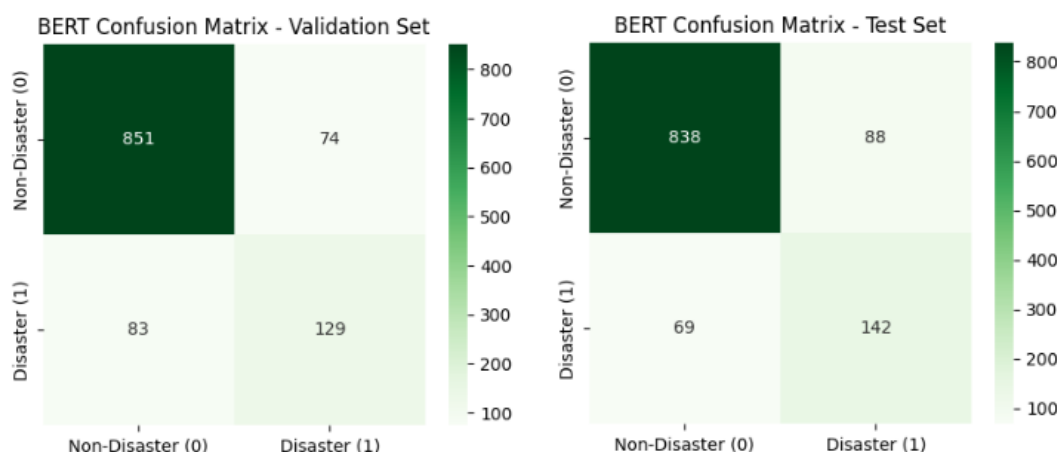
- تعداد پارامترهای قابل آموزش: 1538 (تنها پارامترهای لایه نهایی خطی)

این کار باعث شد تا زمان آموزش به شدت کاهش یابد، احتمال Overfitting روی داده‌های محدود به حداقل برسد و مدل بتواند از دانش زبانی قدرتمند خود که روی داده‌های حجیم دیگری از قبل آموزش دیده بود، برای این مسئله خاص استفاده کند.

پس از آموزش لایه نهایی BERT، مدل روی مجموعه‌های اعتبارسنجی و تست ارزیابی شد. در جدول زیر، نتایج عملکرد هر دو مدل Tiny Transformer (که به طور کامل از صفر آموزش دید) و مدل BERT-based (با رویکرد فریز کردن) روی داده تست مقایسه شده‌اند:

مدل BERT (فریز شده)	مدل Tiny Transformer	معیار ارزیابی
0.8619	0.8347	Accuracy
0.6174	0.5466	Precision
0.6730	0.6398	Recall
0.6440	0.5895	F1-score

همان‌طور که از جدول مشخص است، مدل BERT در تمامی معیارها عملکرد برتری نسبت به Tiny Transformer داشته است. به ویژه در معیار F1-score که نشان‌دهنده تعادل بهتر BERT در تشخیص کلاس اقلیت (توییتهای بحرانی) است.



شکل ۴.۱: ماتریس درهم‌ریختگی عملکرد مدل BERT فریز شده روی داده‌های تست

محدودیت‌های Tiny Transformer Encoder: مدل Tiny که در این پروژه توسعه داده شد، فاقد هرگونه دانش زبانی پیشین است و ناچار بود معنای کلمات، ساختار جمله و وزن Attention میان کلمات را کاملاً از صفر و تنها با مشاهده حدود 7000 توییت بیاموزد. این مدل محدودیت ظرفیتی دارد و منطقاً در درک مفاهیم پیچیده ضعیف‌تر خواهد عمل کرد.

مزایای مدل BERT-based: مدل BERT به دلیل ماهیت پیش‌آموزش‌دیده خود، معنای کلمات را در context جمله درک می‌کند. به عنوان مثال، بعد از دیدن حجم خیلی زیادی از داده، معنای متفاوت کلمه disaster را فهمیده است و میتواند بفهمد در چه جمله‌ای بحران را توصیف میکند و در چه جمله‌ای مثلاً صرفاً یک صفت بد مثل فیلم بد هست.

خلاصه مدل BERT چون زیاد قبلاً آموزش دیده است، با اینکه این مسئله جدید را هنوز ندیده بود، دانش خیلی خوبی کسب کرده بود مثل آدمی هست که درک کامل از زبان دارد و آن موقع به او این توییت‌ها را بدهیم خیلی خوب طبقه‌بندی میکند. البته باید حواسمان باشد که در روش ما چون اکثر وزن‌های شبکه freeze شدند، شبکه از این لحاظ محدود هست یعنی نمیتوان زیاد هم به مسئله خاصی که میخواهیم درونش استفاده کنیم ازش، مدل جهت‌گیری کند و احتمالاً دلیل اینکه نتایجش خیلی بالا مثلاً F1 یک نشد همین است. کلاً برای تسک‌های NLP منطقی هست ولی از fine tuning استفاده کنیم و بجای freeze کردن کل دانش قبلیمان، لایه‌هایی وسطش قابل آموزش بگذاریم آن موقع بهترین نتیجه رو خواهیم گرفت. در اینجا ولی مدل کوچک ما باز نتیجه قابل قبولی داده است نسبی نگاه کنیم و این به ماهیت مسئله برمیگردد، توییت‌های بحرانی حساس به کلمات کلیدی هستند مانند Fire, Earthquake, Emergency. هر چند این کلمات به تنهایی کافی نیستند برای طبقه‌بندی چون در context جمله ممکن است به معنای بحران نباشند ولی یک مدل ترنسفورمر کوچک نیز میتواند با ایجاد وزن‌های توجه قوی روی این توکن‌های خاص، بخش عمده‌ای از توییت‌های بحرانی را به درستی تشخیص دهد.

برای درک عمیق‌تر تفاوت تصمیم‌گیری دو مدل، پیش‌بینی‌های آن‌ها سطر به سطر بررسی شد. نتایج نشان داد در 114 نمونه BERT پاسخ درست و Tiny پاسخ اشتباه داده است، در حالی که در 83 نمونه برعکس این حالت رخ داده است. در ادامه دو نمونه متناقض بررسی می‌شوند:

نمونه اول: برتری BERT

متن: damage emergency closure moray college uhi fully closed staff students tuesday 14th january due storm

برچسب واقعی: 1 (بحران)

پیش‌بینی BERT: 1 (صحیح)

پیش‌بینی Tiny: 0 (اشتباه)

در این توییت از آسیب، تعطیلی اضطراری و طوفان صحبت شده است. مدل Tiny احتمالاً نتوانسته ارتباط معنایی بین کلمات دور از هم را برقرار کند و با دیدن کلمات روزمره‌ای چون college, staff و students وزن بیشتری به کلاس غیربحرانی داده است. اما BERT با استفاده از دانش زبانی عمیق خود، ترکیب ”تعطیلی اضطراری” و ”طوفان” را به عنوان یک رویداد بحرانی واقعی استنتاج کرده است.

نمونه دوم: برتری Tiny

متن: military trump divert extra 7 2 billion military border wall reports say

برچسب واقعی: 0 (خبر سیاسی/غیربحرانی)

پیش‌بینی BERT: 1 (اشتباه)

پیش‌بینی Tiny: 0 (صحیح)

این توییت یک خبر سیاسی دربارهٔ بودجه است و هیچ بحرانی را گزارش نمی‌کند. مدل BERT به احتمال زیاد به دلیل وجود کلماتی مانند military و border wall که معمولاً با درگیری‌ها و تنش‌ها همراه هستند، دچار خطای False Positive شده است. در مقابل، مدل Tiny چون کلا ساده تر هست، احتمالاً زیاد تحلیل نکرده است که این کلمات چطور میتواند به بحران ربط پیدا کند (ولی BERT چون داده بیشتر دیده است، درک بیشتری از این کلمات دارد) و همین سادگی باعث شده بگوید کلمه رایج فجایع طبیعی که در دیتاست کوچک خودم بود را ندیدم و به درستی آن را کلاس 0 طبقه‌بندی کرده است.

۵.۲.۱ بخش امتیازی

بررسی اثر حذف Positional Encoding

برای بررسی نقش اطلاعات موقعیتی در مدل‌های مبتنی بر ترنسفورمر، لایه Positional Encoding از معماری Tiny Transformer حذف شد و مدل مجدداً با همان تنظیمات (Hyperparameters) آموزش دید.

در جدول زیر، نتایج ارزیابی مدل روی داده تست در دو حالت با و بدون جایگذاری اطلاعات موقعیتی مقایسه شده است:

تغییرات	بدون Positional Encoding	با Positional Encoding (مدل اصلی)	معیار ارزیابی (Test Set)
0.0132	0.8215	0.8347	Accuracy
0.0307	0.5159	0.5466	Precision
0.0237	0.6161	0.6398	Recall
0.0279	0.5616	0.5895	F1-score

همانطور که میبینیم در تمام متریک‌ها داشتن Positional Encoding بهتر هست ولی با حذفش افت خیلی شدید هم نمیکنیم و این برمیگردد به ماهیت مسئله طبقه‌بندی متن در توییت‌های بحران است. در بسیاری از موارد، صرف حضور کلماتی مانند ”آتش‌سوزی”، ”زلزله” برای تشخیص بحران کافی

است و ترتیب آن‌ها نقش ثانویه دارد.

بررسی اثر تغییر تعداد Headها

هر دو مدل Tiny Transformer (آموزش از صفر) و BERT با تنظیمات 2 هد و 8 هد آموزش داده شدند و عملکرد آن‌ها روی مجموعه تست ارزیابی شد.

مدل	تعداد Head	Accuracy	Precision	Recall	F1-score
Tiny Transformer	2	0.8470	0.5920	0.5640	0.5777
Tiny Transformer	8	0.8232	0.5184	0.6682	0.5839
(Fine-tuned) BERT	2	0.8056	0.4868	0.8720	0.6248
(Fine-tuned) BERT	8	0.8962	0.7360	0.6872	0.7108

در اینجا قشنگ میشود بحث پراکندگی توجه در مقابل عمق بیشتر در یک جهت را دید که اگر ظرفیت مدل زیاد باشد یعنی توجه زیاد دارد که تقسیم کند و هر کدام را بیشتر بررسی کند ولی برای مدل ساده، تعداد دیدگاه‌ها را بیشتر کردن باعث میشود هر کدام توجه خیلی کمی داشته باشند چون کلاً مدل ظرفیت کمتری دارد.

در مدل Tiny که از صفر آموزش می‌بیند، افزایش تعداد هدها از 2 به 8 نه تنها دقت کل را بهبود نداد، بلکه باعث افت بیش از 2 درصدی آن شد و همچنین افت شدید Precision شد. وقتی بعد Embedding مدل تنها 64 است، در حالت 8 هدی، سهم هر هد تنها 8 بعد خواهد بود. در این فضای بسیار کوچک و با توجه به محدودیت شدید داده‌های آموزشی، مدل نمی‌تواند الگوهای معناداری در 8 فضای مجزا پیدا کند. در واقع، مدل دچار پراکندگی توجه می‌شود. در این سناریو، استفاده از 2 هد (با فضای 32 بعدی برای هر کدام) به مدل اجازه داده است تا بازنمایی‌های پایدارتری بسازد.

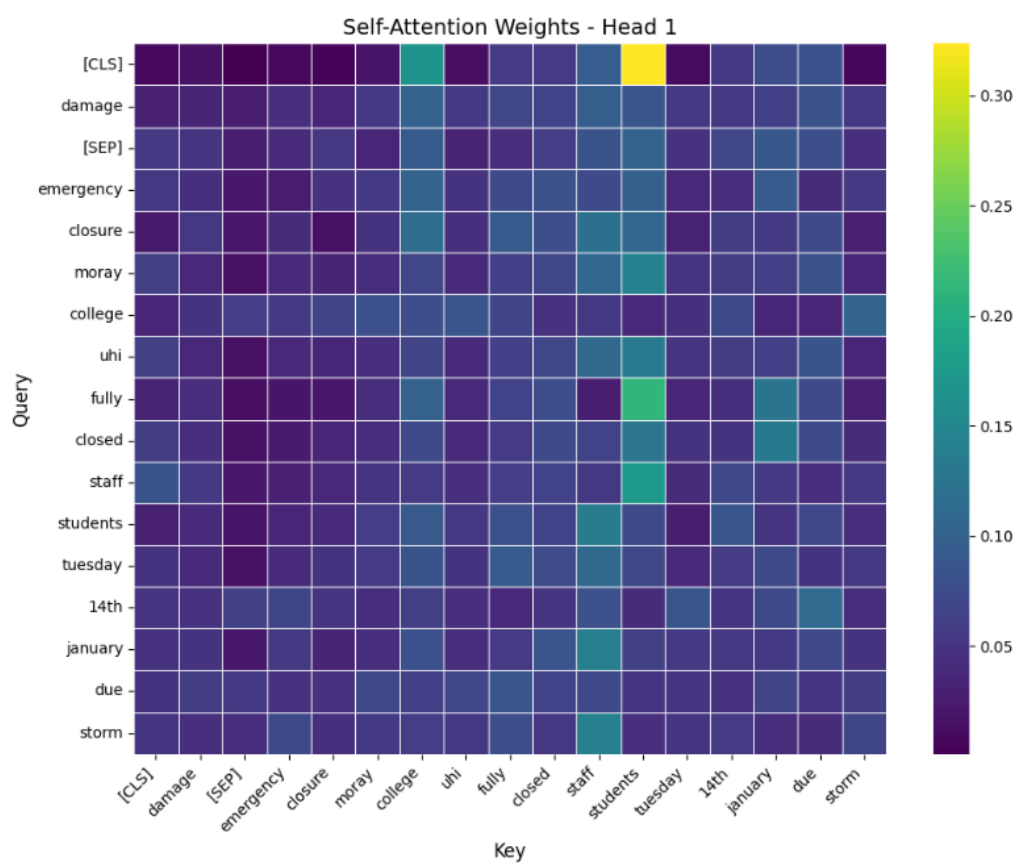
رفتار مدل BERT کاملاً برعکس است. با افزایش تعداد هدها از 2 به 8، دقت مدل به شکل چشمگیری (از 80.5% به 89.6%) و F1-score نیز ارتقای قابل توجهی یافته است. مدل BERT با 2 هد دچار یک سوگیری شدید شده بود؛ به طوری که Recall بسیار بالا (0.87) و Precision بسیار پایین (0.48) داشت (یعنی اکثر توییت‌ها را به اشتباه بحران تشخیص می‌داد). اما با تخصیص 8 هد، مدل توانست وظایف توجه را تخصصی‌تر کند. از آنجا که BERT دارای ابعاد پنهان بزرگ و دانش زبانی از پیش‌آمخته شده است، به خوبی از افزایش تعداد Headها برای درک تفاوت‌های مهم در ساختار جملات استفاده کرده و تعادل بهتری بین Precision و Recall بوجود آورد.

تحلیل وزن‌های Attention

در این بخش، ماتریس Attention مدل Tiny Transformer برای یکی از توییت‌های مجموعه آزمون استخراج و به صورت Heatmap رسم شد. متن بررسی شده: [CLS] damage [SEP] emergency closure moray college uhi fully closed staff students tuesday 14th january due storm

در نمودار رسم شده، محور افقی نشان‌دهنده Key (کلمه‌ای که اطلاعات می‌دهد) و محور عمودی نشان‌دهنده Query (کلمه‌ای که در حال پردازش است) می‌باشد. شدت رنگ نشانگر میزان توجه است.

از آنجا که در معماری ما، بردار خروجی توکن [CLS] به عنوان نماینده کل متن به لایه Classifier داده می‌شود، سطر مربوط به این توکن حیاتی‌ترین



شکل ۵.۱: نقشه حرارتی توزیع وزن‌های Attention مدل Transformer Tiny

بخش ماتریس است. توکن [CLS] بالاترین میزان توجه خود (رنگ زرد روشن) را به کلمه students و پس از آن به کلمه college اختصاص داده است. کلماتی که نشان‌دهنده یک فاجعه واقعی هستند نظیر storm، emergency و damage، در سطر [CLS] کاملاً تیره (وزن نزدیک به صفر) هستند و مدل هیچ توجهی به آن‌ها نکرده است.

دقیقاً علت خطای مدل Tiny Transformer را در پیش‌بینی این نمونه (که به اشتباه کلاس 0 یا غیربحرانی تشخیص داده شد) در اینجا دیده میشود. از آنجا که مدل روی یک دیتاست کوچک و بدون دانش قبلی آموزش دیده است، احتمالاً در داده‌های آموزشی کلماتی مثل students و college غالباً در توییت‌های روزمره و غیربحرانی به کار رفته‌اند. در نتیجه، مدل دچار یک سوگیری شده و به جای درک معنای کلی جمله و ارتباط تعطیلی با طوفان، صرفاً با دیدن کلمه students، غیر بحرانی تشخیص داده است.